

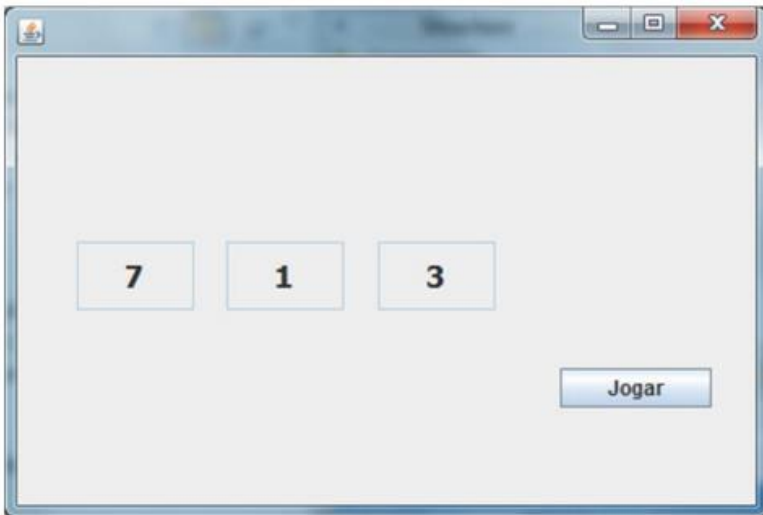
LISTA DE EXERCÍCIOS - THREADS

Para responder à questão abaixo, considere o arquivo Threads.pdf

1. Compare os dois programas nas transparências 5 e 8. Quais diferenças você identifica nos códigos entre estes programas? Explique estas diferenças.
2. Crie um programa em interface texto (via console) que receba duas matrizes de inteiros de tamanho $N \times N$ e imprima a soma das matrizes. A soma das matrizes, porém, deve ser paralelizada em quatro threads, de forma que cada thread é responsável por somar um quadrante diferente da matriz.

Para responder às questões abaixo, considere o arquivo TestaThread.java

3. Encontre o método “`SwingUtilities.invokeLater`”, por que ele é usado?
4. A chamada do método “`pBar.setValue(cont);`” na linha 22, altera a representação gráfica da `JProgressBar`. Explique por que o esta chamada não precisa do método “`SwingUtilities.invokeLater`”.
5. Poderíamos executar “`pBar.setValue(cont++);`” da linha 31 sem usar o “`SwingUtilities.invokeLater`”? Explique.
6. É possível substituir o “`SwingUtilities.invokeLater`” pelo método “`SwingUtilities.invokeAndWait`”? Apresente as alterações necessárias no programa. Qual a diferença entre estes métodos?
7. Implemente um programa com o Java Swing que simule uma aplicação de caça-níquel, conforme a figura abaixo (o design da janela fica de livre escolha do aluno):



O caça níquel tem 3 `JTextFields`, independentes, que giram, aleatoriamente, de 1 a 150 vezes, variando entre os números de 1 a 7, durante um certo período de tempo. Quando todos os `JTextFields` pararem de girar, abra uma caixa de diálogo dizendo que o usuário venceu o jogo, se os três números forem iguais, ou que perdeu, caso contrário. O `JButton` “Jogar” deve ficar desabilitado enquanto os números estiverem rodando.